

Informe — Forma K: Cursos y Matrículas

Grupo: 05

Alan Ibacache

Benjamín Jiménez

Benjamín Villarroel

Docente: Gonzalo Pérez Correa

Facultad de Ingeniería — Universidad de Concepción

25 de septiembre de 2026

1. Problema y solución

1.1. Contexto

El Centro de Formación Técnica AprendeMás opera hoy con dos sistemas que nacieron por separado y nunca conversaron entre sí:

- **Cupos:** administra los cursos y los cupos disponibles de cada uno. Es donde vive la verdad sobre qué cursos se dictan y cuántos cupos quedan en cada uno.
- **Matrículas:** registra a los estudiantes y sus matrículas. Necesita saber, para cada curso, si hay cupos disponibles antes de matricular a un estudiante, y consulta esa información con muchísima frecuencia.

Hoy los dos sistemas no están integrados: se matricula sin verificar cupos y algunos cursos superan la capacidad de la sala. La organización encargó una solución de integración que resuelva este problema.

1.2. Restricciones y lo que no es evidente

Del enunciado se desprenden restricciones que condicionan el diseño más allá de “conectar dos sistemas”:

1. **La decisión de cupo debe tomarla el dueño del dato.** Si Matrículas consultara la disponibilidad y luego registrara la matrícula con esa copia, dos inscripciones simultáneas podrían ver el mismo último cupo. La verificación y el descuento deben ocurrir juntos, de forma atómica, dentro de Cupos.
2. **“Consulta con muchísima frecuencia” no implica decidir con datos cacheados.** Una caché acelera las lecturas de disponibilidad, pero si se usara para autorizar matrículas reintroduciría el sobrecupo. Hay que separar la lectura informativa de la decisión.
3. **La red falla a mitad de una operación.** Un cliente que pierde la respuesta de una matrícula reintentará; sin protección, ese reintento ocupa un segundo cupo.

4. **Cupos puede no estar disponible.** La API debe responder algo definido y rápido, sin matricular a ciegas.
5. **Hay datos personales.** Matrículas guarda nombres y correos de estudiantes; Cupos solo guarda la oferta académica.

1.3. Solución

La solución tiene dos servicios: **Matrículas** gestiona estudiantes y matrículas y expone una API REST pública; **Cupos** gestiona los cursos y expone un servicio gRPC interno. Cada uno tiene su propia base SQLite. Redis, usado solo por Matrículas, entrega caché de disponibilidad e idempotencia. Al recibir una solicitud de matrícula, la API REST invoca `ocuparCupo` en Cupos, que verifica y descuenta el cupo en una sola operación atómica; solo si Cupos confirma se registra la matrícula. Así se cierra la brecha que hoy permite matricular sin cupo.

Las decisiones de fondo están documentadas como ADR (*Architecture Decision Records*) en `docs/adr/` y en el Anexo A: la frontera entre servicios (ADR-001), el protocolo hacia cada lado (ADR-002), la estrategia de evolución del contrato (ADR-003), el manejo de fallas (ADR-004) y la autenticación (ADR-005). Este informe explica esas decisiones en conjunto, qué otras opciones eran viables, por qué se descartaron y qué se midió para respaldarlas.

2. Arquitectura

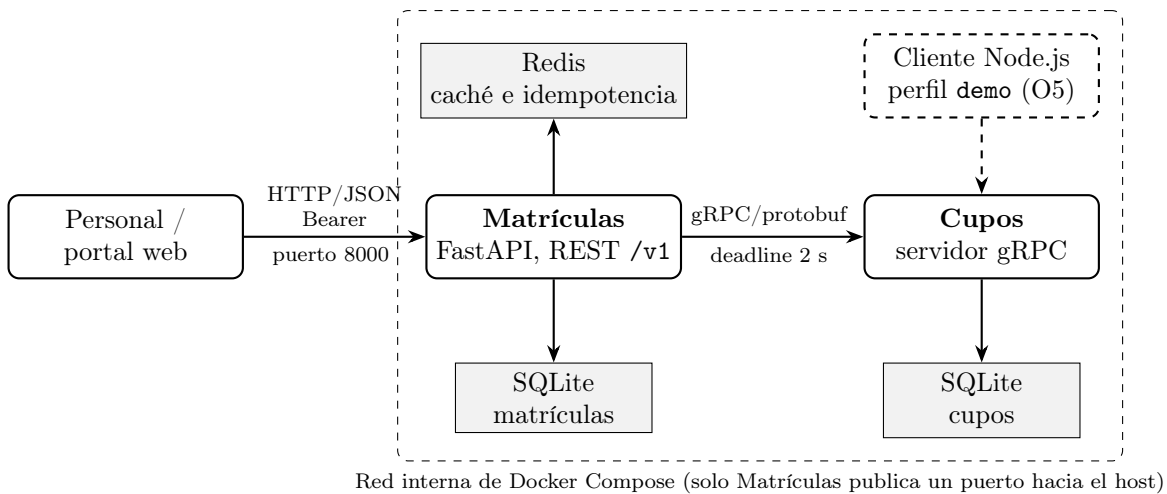


Figura 1: Arquitectura de la solución. Cada servicio es dueño de su base; ninguno accede a la del otro.

- **Un solo punto de entrada.** Solo Matrículas publica un puerto (8000). Cupos (50051) y Redis (6379) únicamente son alcanzables dentro de la red de Docker Compose, coherente con que la comunicación con Cupos es interna.
- **Contratos como fuente única.** `matriculas/openapi.yaml` se escribe a mano (*contract-first*) y es exactamente lo que la API publica en `/openapi.json`, `/openapi.yaml` y `/docs`. `cupos/cupos.proto` es el único `.proto` del repositorio: los stubs de Python se generan desde él al construir cada imagen y el cliente Node lo carga directamente. El código generado no se versiona, para que ningún consumidor quede con una copia desactualizada del contrato.

- Todo se levanta con un comando: `docker compose up --build` (ver §11).

3. Estilo arquitectónico: microservicios

Decisión (ADR-001): dos servicios independientes, cada uno con su propio proceso, base de datos y ciclo de despliegue, divididos por *bounded context* [4, 2]: Cupos posee **Curso** y sus contadores; Matrículas posee **Estudiante** y **Matrícula**.

3.1. Alternativas consideradas

Opción	Qué ofrece	Por qué no se eligió
Monolito (todo en un proceso, una base de datos)	Transacción local simple, sin llamadas de red, cero costo de coordinación.	Acopla a los dos equipos y ciclos de vida que hoy son distintos: Cupos cambia con la oferta académica, Matrículas con las reglas de inscripción. Un solo despliegue frenaría a ambos. Es una opción legítima si un solo equipo mantuviera ambos dominios, pero no resuelve el problema real, que es la falta de comunicación entre dos sistemas que ya existen por separado .
Monolito en capas (presentación / lógica / datos, un solo despliegue)	Organización interna clara, sigue siendo una sola unidad desplegable.	Mismo problema que el monolito simple desde el punto de vista de integración: resuelve el eje “cómo se organiza por dentro”, no el eje “cómo se comunican dos sistemas”, que es el problema que el encargo plantea.
Arquitectura hexagonal dentro de un único servicio	Aísla el núcleo de negocio de la tecnología de persistencia o transporte.	Es un patrón de organización interna (eje 1), no de descomposición entre sistemas (eje 2). Podría aplicarse dentro de cada microservicio, pero no reemplaza la necesidad de separar Cupos de Matrículas.
Dos servicios con base de datos compartida	Evita la llamada remota para leer datos del otro dominio; consultas SQL directas y JOIN entre tablas.	Es el anti-patrón “monolito distribuido”: se pagan los costos de la distribución (dos despliegues, dos procesos) sin ganar autonomía real, porque un cambio de esquema en un servicio puede romper al otro sin que medie ningún contrato.
Microservicios con base de datos por servicio (elegida)	Autonomía real: cada servicio evoluciona su esquema sin coordinar con el otro; el único acceso a los datos ajenos es a través de la API/contrato del dueño.	Cuesta una llamada remota para cruzar información y renuncia a una transacción ACID entre ambos dominios (ver §3.3).

Tabla 1: Alternativas arquitectónicas consideradas.

3.2. Datos duplicados a propósito

- `curso_id` dentro de cada matrícula: es el único identificador compartido entre los dos contextos.
- Una copia de la disponibilidad en Redis, con TTL de 30 s, usada solo por `GET /v1/cursos/{id}`. Puede estar desactualizada hasta 30 s; por eso **nunca** se usa para decidir una matrícula (restricción 2 de §1.2): esa decisión siempre la toma `Cupos` con `OcuparCupo`.

3.3. Costo aceptado

La matrícula cruza la red: puede ocurrir que se ocupe un cupo en `Cupos` y la escritura local en `Matrículas` falle después. Se acepta ese riesgo en vez de introducir una transacción distribuida (2PC) o un mecanismo de compensación tipo Saga, porque el volumen y la criticidad del dominio no lo justifican en esta etapa; una versión de producción debería agregar un identificador de operación y un proceso de conciliación periódica.

Hay dos mitigaciones puntuales, sin llegar a una compensación general:

- **Matrícula duplicada concurrente.** Un índice único parcial en SQLite impide dos matrículas *activas* del mismo estudiante en el mismo curso. Si dos peticiones simultáneas llegan a ocupar cupo, la segunda falla al insertar y `Matrículas` devuelve el cupo con `LiberarCupo` antes de responder 409.
- **Reversión concurrente.** Al revertir, primero se marca la matrícula como **revertida** con un `UPDATE` condicional y recién después se libera el cupo; si la liberación falla, la matrícula vuelve a **activa**. Dos reversiones simultáneas no pueden liberar el cupo dos veces.

4. Estilo de integración: invocación remota

El catálogo de Enterprise Integration Patterns [1] define cuatro estilos para que dos sistemas colaboren. El proyecto usa **invocación remota**; los otros tres se descartaron por razones concretas del dominio:

Estilo de integración	Por qué no aplica aquí
Transferencia de archivos (ej. batch nocturno de cupos)	La disponibilidad de cupos cambia con cada matrícula; un archivo periódico dejaría a Matrículas trabajando con datos obsoletos y permitiría sobrecupos, exactamente el problema que el encargo pide resolver.
Base de datos compartida	Ya descartada en §3.1: rompe la propiedad de los datos y crea acoplamiento de esquema entre dos equipos que deben poder evolucionar por separado.
Mensajería (cola o tópico con broker)	Es la respuesta natural cuando la relación es “avísame cuando algo cambie” y se tolera consistencia eventual. Aquí la pregunta es “¿hay cupo <i>ahora mismo</i> , antes de confirmar esta matrícula?”: es una decisión síncrona que bloquea o permite una operación de negocio, no un evento que se pueda propagar con demora. Mensajería es contenido de la Unidad 2 del curso (asíncrona) y queda fuera del alcance de este encargo, que trabaja explícitamente sobre comunicación síncrona.
Invocación remota (elegida)	Matrículas necesita una respuesta inmediata de Cupos para decidir si matricula o rechaza. Es el estilo correcto cuando la operación es una pregunta-respuesta que debe resolverse en el momento.

Tabla 2: Evaluación de estilos de integración.

Dentro de invocación remota, también se descartó explícitamente el patrón **SOA con bus (ESB)**: exigiría una gobernanza centralizada, contratos WSDL/SOAP y un equipo que administre el bus, todo pensado para organizaciones con decenas de sistemas heredados e inmodificables. AprendeMás tiene dos sistemas propios y un equipo que los controla por completo — exactamente el contexto donde microservicios rinde mejor que SOA [3], no al revés.

5. REST hacia afuera, gRPC hacia adentro

Decisión (ADR-002): la API pública de Matrículas es REST/JSON bajo /v1; la comunicación Matrículas → Cupos es gRPC/Protocol Buffers [11].

5.1. Alternativas consideradas

Opción	A favor	En contra
REST/JSON en ambos lados	Un solo protocolo que aprender y depurar; cualquier cliente HTTP sirve, incluso para la llamada interna.	Sin contrato compilado: un error de tipo o de nombre en el JSON pasa en silencio hasta que el servidor interpreta mal el dato. Sin streaming nativo. En nuestro experimento (§10.1) el mensaje <code>Curso</code> pesó 94 bytes en JSON contra 39 en protobuf; con gzip la diferencia se reduce, pero no desaparece.
gRPC en ambos lados	Contrato tipado y generado también hacia afuera; más eficiente en el borde también.	El navegador no puede hablar gRPC nativamente (no expone el framing binario de HTTP/2 ni los trailers donde viaja el código de estado); expondría un servicio interno de alto volumen directamente a clientes externos, mezclando dos superficies de confianza distintas.
REST hacia afuera, gRPC hacia adentro (elegida)	Cada protocolo se usa donde su costo se paga con una ventaja real: interoperabilidad en el borde, contrato tipado y eficiencia en el tráfico interno de alto volumen.	Duplica la superficie de mantenimiento: dos contratos (<code>openapi.yaml</code> y <code>cupos.proto</code>), dos formatos, dos juegos de herramientas de depuración.

Tabla 3: Protocolos de comunicación interna y externa.

Dentro de gRPC, `ListarCursos` usa **server streaming** en vez de una llamada unaria, porque el catálogo de cursos puede crecer y el criterio del curso es “si el resultado es grande o se produce con el tiempo, streaming; si es una respuesta corta y completa, unaria”. `ConsultarCurso`, `OcuparCupo` y `LiberarCupo` sí son unarios, porque cada uno resuelve una sola pregunta o una sola escritura atómica. Se descartó **client streaming** y **bidireccional** por no existir ningún caso de uso en este dominio que envíe muchos mensajes hacia Cupos o que necesite un canal abierto en ambos sentidos (eso correspondería a un chat o una negociación por rondas, no a matricular estudiantes).

El puerto de Cupos no se publica fuera de la red de Compose. Si en el futuro un cliente externo necesitara streaming, la vía sería **gRPC-Web con un proxy**, no abrir el puerto interno — así queda anotado en el ADR-002 como decisión pendiente y no como omisión.

6. Patrones implementados y alternativas descartadas

Necesidad	Patrón elegido	Cómo se implementó	Alternativas descartadas y por qué
Evitar consultar a Cupos en cada lectura de disponibilidad	Cache-aside con Redis	GET /v1/cursos/{id} busca primero en Redis; si no está, consulta a Cupos por gRPC y guarda el resultado con TTL de 30 s; ocupar o liberar un cupo invalida la clave. Si Redis falla, la lectura va directo a Cupos y se deja de intentar Redis durante 15 s: la caché es una optimización, no una dependencia. La matrícula nunca decide con la caché.	<i>Sin caché</i> : cada consulta golpea a Cupos por gRPC, lo cual es correcto pero más lento bajo carga. <i>Write-through</i> : actualizar la caché en cada escritura de Cupos en vez de invalidar desde Matrículas — se descartó porque acoplaría a Cupos con la existencia de una caché en un consumidor suyo, violando la frontera de servicio.
Evitar que un reintento de red duplique una matrícula	Idempotent Receiver vía Idempotency-Key	El cliente envía un identificador único por intención de matrícula. La clave se reserva en Redis con SET NX [15], que es atómico: de dos reintentos simultáneos solo uno procesa y el otro recibe 409. El resultado se guarda 24 h y un reintento posterior recibe la matrícula original (200, Idempotent-Replay: true) sin ocupar otro cupo. Si la matrícula falla, la clave se libera para reintentar. Sin Redis, una petición con clave recibe 503 en vez de arriesgar un duplicado.	<i>Sin idempotencia</i> (dejar que el cliente no reintente): no resuelve el problema real, porque el cliente puede perder la respuesta sin saber si la operación se ejecutó. <i>Deduplicar por contenido</i> (mismo estudiante y curso): no distingue un reintento, que debe devolver la matrícula original con 200, de un segundo intento real, que debe rechazarse con 409. Por eso son dos mecanismos separados: la clave para los reintentos y la regla de “una matrícula activa por curso” para el negocio.
Comunicar qué acciones son válidas desde el estado actual	HATEOAS [5]	La respuesta de una matrícula incluye _links ; el enlace revertir solo aparece si el estado es activa .	<i>Documentar las transiciones solo en el OpenAPI</i> (sin enlaces en la respuesta): es más liviano y es lo que se usa en el resto de la API (estudiantes, cursos), pero para matrículas se prefirió HATEOAS porque el estado de una matrícula cambia (activa → revertida) y el enlace evita que el cliente necesite conocer de memoria qué transición es válida. La validación real igual ocurre en el servidor: revertir dos veces responde 409.

Necesidad	Patrón elegido	Cómo se implementó	Alternativas descartadas y por qué
Formato uniforme de errores	Problem Details (RFC 9457) [7]	Todo error responde <code>application/problem+json</code> con <code>type</code> , <code>title</code> , <code>status</code> y <code>detail</code> , incluidos los 404/405 de rutas inexistentes, los 422 de validación (con la lista <code>errors</code>) y los 500 inesperados. Los mensajes no incluyen trazas ni direcciones internas.	<i>Formato propio</i> (un objeto JSON con un campo <code>error</code>): más simple de escribir, pero cada API termina inventando su propia forma, y un cliente que consume varias APIs debe aprender un formato distinto por cada una. RFC 9457 es un estándar existente y evita esa fragmentación.
Evolucionar el contrato sin romper consumidores	Versionado por ruta (<code>/v1</code>) y contract-first en REST; evolución aditiva en protobuf	El <code>openapi.yaml</code> escrito a mano es el contrato publicado. Un cambio incompatible crearía <code>/v2</code> ; en protobuf se agregan campos con números nuevos y los eliminados se marcan <code>reserved</code> . Detalle en el ADR-003.	<i>Versionado por cabecera</i> (<code>Accept: ...;version=1</code>): más “puro” porque la URL identifica solo el recurso, pero invisible en el navegador y más difícil de depurar en la defensa oral. <i>Sin versión, solo cambios aditivos</i> : exige una disciplina muy alta y un mecanismo de deprecación que el equipo no tiene la madurez operacional para sostener en un proyecto de este tamaño.
Entregar colecciones que crecen	Paginación por desplazamiento	<code>limit</code> (20 por defecto, máximo 100) y <code>offset</code> ; la respuesta trae <code>items</code> , <code>total</code> , <code>limit</code> y <code>offset</code> .	<i>Devolver la colección completa</i> : funciona con diez registros y degrada el servicio con miles; agregar paginación después cambia la forma de la respuesta, un cambio incompatible. <i>Paginación por cursor</i> : estable ante inserciones y más eficiente en tablas grandes, pero más compleja; el volumen de un CFT no la justifica aún.

Necesidad	Patrón elegido	Cómo se implementó	Alternativas descartadas y por qué
Responder ante una dependencia caída o lenta	Fail-fast con deadline	El canal gRPC usa un deadline de 2 s [12]; UNAVAILABLE → 503, DEADLINE_EXCEEDED → 504, ambos reintentables con Idempotency-Key. La secuencia real al caer Cupos se midió en §10.2.	<i>Esperar sin deadline</i>: maximiza el número de matrículas que terminan bien, pero agota hilos/conexiones si Cupos está lento, arrastrando a Matrículas con él. <i>Aceptar con la última caché conocida</i>: mantiene el servicio disponible, pero puede autorizar una matrícula sobre un cupo que ya no existe — es precisamente el sobrecupo que el encargo pide evitar. Se prefirió sacrificar disponibilidad temporal antes que la corrección del dato de cupos.
Identificar al llamador y separar permisos	Bearer con tokens cos por rol [8]	Token de escritura (todas las operaciones) y token de solo lectura (solo GET). Sin token o con uno desconocido: 401 con WWW-Authenticate: Bearer. Token válido que no alcanza (lectura intentando un POST): 403. Detalle en el ADR-005.	<i>Basic Auth</i>: viaja usuario y contraseña en cada petición y exige un almacén de contraseñas que el sistema no tiene. <i>API Key en un header propio</i>: igual de simple, pero no es un esquema HTTP estándar ni define cómo informar por qué se rechazó. <i>JWT</i>: identidad, roles y expiración sin consultar a nadie, pero exige gestionar claves y emisión de tokens; queda como evolución (§13.2).

7. Flujo de una matrícula y modos de falla

Al recibir POST /v1/matriculas, Matrículas:

1. Si viene Idempotency-Key, la reserva en Redis; si ya tenía resultado, lo devuelve (200) sin hacer nada más.
2. Verifica que el estudiante exista (404 si no) y que no tenga ya una matrícula activa en ese curso (409).
3. Invoca OcuparCupo en Cupos, que verifica y descuenta el cupo bajo un lock, en una sola operación. Si no hay cupos, Cupos responde FAILED_PRECONDITION y la API, 409.

4. Persiste la matrícula como **activa**, invalida la caché del curso, guarda el resultado para la clave de idempotencia y responde 201 con cabecera **Location**.

Situación	HTTP	Comportamiento
Cupos detenido, primeros 20 s	504	Cada llamada espera el deadline de 2 s (§10.2).
Cupos detenido, después de 20 s	503	Falla inmediata (~9 ms).
Cupos responde un error inesperado	502	Mensaje fijo, sin detalles internos.
Curso inexistente / sin cupos	404 / 409	Mensaje de negocio de Cupos.
Redis caído, lectura de curso	200	Se consulta a Cupos sin caché.
Redis caído, matrícula con Idempotency-Key	503	No se puede garantizar la idempotencia; no se ocupa cupo.
Redis caído, matrícula sin clave	201	Se procesa normalmente.
Redis caído, latencia	—	La primera petición tarda ~8 s (resolver el nombre de un contenedor detenido); luego se omite Redis por 15 s y se responde en 25–40 ms.
Error no previsto en Matrículas	500	problem+json sin traza.

Tabla 5: Modos de falla y respuesta de la API. Todos verificados con el sistema levantado en Docker.

8. Seguridad y contexto: privacidad de los datos personales

El factor de contexto elegido es la **privacidad de los datos de estudiantes** (nombres y correos). No quedó como una recomendación final: moldeó decisiones concretas del diseño:

- **Frontera y bases separadas** (§3): los datos personales existen solo en la base de Matrículas. Cupos guarda oferta académica, que es pública. Una base compartida habría ampliado la superficie de exposición sin necesidad.
- **Superficie de red mínima**: solo la API publica un puerto; Cupos y Redis no son alcanzables desde fuera de la red de Compose.
- **Mínimo privilegio**: el token de solo lectura permite consultar sin poder matricular ni revertir.
- **Nada personal en Redis**: la caché guarda solo cursos y las respuestas de idempotencia contienen identificadores (**estudiante_id**, **curso_id**), no nombres ni correos.
- **Errores sin detalles internos**: las respuestas de error no exponen trazas ni direcciones de la red interna.

Lo que falta para producción está en §13.2: TLS, identidades personales (JWT) para auditar quién hizo cada operación, y una política de retención de datos.

9. Requisitos opcionales

- **O1 — Caché con Redis**: cache-aside con TTL de 30 s e invalidación al ocupar o liberar (§6).
- **O2 — Idempotencia**: Idempotency-Key con reserva atómica y resultado guardado 24 h (§6).

- **O3 — HATEOAS:** `_links` según el estado de la matrícula (§6).
- **O4 — Pruebas de contrato:** `tests/test_api.py` ejecuta la API real, con Cupos y Redis simulados, y valida cada respuesta contra `openapi.yaml`: que el código de estado esté declarado para esa operación y que el cuerpo cumpla el esquema, con `additionalProperties: false`. Se verificó que las pruebas fallan si se agrega a una respuesta un campo no declarado o si se responde 403 donde corresponde 401. `tests/test_contracts.py` verifica además que el `.proto` exponga las cuatro operaciones y que `ListarCursos` sea streaming.
- **O5 — Segundo cliente gRPC:** cliente Node.js (§9.1).

9.1. Segundo cliente gRPC e interoperabilidad (O5)

Para evidenciar la interoperabilidad del contrato binario, se desarrolló un cliente independiente en Node.js que invoca las cuatro operaciones de Cupos cargando el mismo `cupos/cupos.proto` que usa el servidor, sin copia propia y sin modificar el servidor Python. Como Cupos no publica su puerto, el cliente corre como un contenedor más dentro de la red de Compose (perfil `demo`).

Hallazgo técnico y resolución. En las primeras pruebas, el cliente Node.js no encontraba ciertos atributos retornados por el servidor. La compatibilidad a nivel de red (*wire format*) estaba intacta, dado que Protocol Buffers identifica los campos por su número (*field tag*) y no por su nombre [11]. El problema estaba en la capa de deserialización de la librería de Node.js (`@grpc/proto-loader`), que por convención de JavaScript convierte los nombres de `snake_case` a `camelCase` (`curso_id` pasa a `cursoId`). La solución fue configurar `keepCase: true` al cargar el descriptor, respetando los nombres del contrato sin alterar una sola línea del `.proto`.

Conexión con el ADR-003. El hallazgo confirma que el contrato es independiente del lenguaje consumidor y que las diferencias idiomáticas entre ecosistemas se resuelven en la configuración del cliente, no modificando el contrato central.

10. Experimentos

Los dos experimentos son reproducibles con los scripts del repositorio; los datos crudos están en `docs/resultados/`.

10.1. Tamaño y costo de serialización: protobuf frente a JSON

Hipótesis. Un `Curso` serializado en protobuf ocupa menos bytes que su equivalente JSON y se serializa más rápido; con compresión gzip la diferencia de tamaño se reduce.

Método. `python docs/experimento_serializacion.py`. (a) Se serializa el mismo `Curso` (ARQ-101, 4 campos) diez series de 10.000 veces con cada formato, reportando tamaño, media y desviación estándar por operación. JSON se genera sin espacios entre separadores, para no inflarlo artificialmente. (b) Se comparan listas de $n = 1, 10, 100, 1000$ cursos, sin y con gzip. Para protobuf se suma el tamaño de cada mensaje, porque `ListarCursos` envía un mensaje por curso; no se cuentan los 5 bytes de encabezado que gRPC agrega a cada mensaje. Entorno: Python 3.14.2, protobuf 7.36.2, Windows 11.

Resultados. Un Curso: **39 bytes en protobuf frente a 94 en JSON** (0,41 veces el tamaño), idéntico en todas las ejecuciones. El tiempo por operación varió entre ejecuciones: en cinco ejecuciones, protobuf tomó entre 236 y 542 ns/op y JSON entre 3.760 y 5.171 ns/op. JSON fue entre 9,5 y 18 veces más lento en todas.

n	protobuf	JSON	pb/JSON	pb + gzip	JSON + gzip	pb/JSON con gzip
1	34	92	0,37	52	97	0,54
10	358	911	0,39	128	176	0,73
100	3.684	9.261	0,40	654	766	0,85
1.000	37.840	93.641	0,40	6.129	7.521	0,81

Tabla 6: Tamaño en bytes de n cursos según formato, sin y con gzip.

Conclusiones y límites.

- Sin compresión, protobuf ocupa de forma estable **el 40 % de JSON**, independiente de n : el ahorro viene de no repetir los nombres de los campos en cada elemento.
- **Con gzip la ventaja se reduce a 0,73–0,85**, porque gzip elimina justamente esa repetición. En un mensaje suelto gzip *agrand*a ambos formatos (su encabezado pesa más que lo que ahorra), así que comprimir mensajes pequeños no conviene.
- El tiempo de serialización (del orden de microsegundos) es despreciable frente a la latencia de red y al deadline de 2 s. **El rendimiento no es el argumento decisivo** para usar gRPC adentro; lo es el contrato tipado y generado (ADR-002).
- Límites: los nombres de los cursos del inciso (b) son sintéticos y repetitivos, lo que favorece a gzip. Los tiempos dependen de la carga de la máquina: la variación entre ejecuciones muestra que no se controló. Este proyecto no configura compresión en gRPC.

10.2. Resiliencia: qué responde la API cuando Cupos cae

Hipótesis. Con el deadline de 2 s, cuando Cupos no está disponible la API responde rápido con 503 y nunca tarda más que el deadline.

Método. `python docs/experimento_resiliencia.py --repeticiones 5 --duracion 35`, con el sistema levantado en Docker. En cada repetición:

1. Se espera a que la API responda con Cupos arriba.
2. Se detiene el contenedor de Cupos (`docker compose stop cupos`).
3. Durante 35 s se consulta `GET /v1/cursos/NO-EXISTE` una vez por segundo, registrando código y latencia.
4. Se reinicia Cupos y se mide cuánto tarda la API en volver a responder normalmente.

Se consulta un curso inexistente a propósito: con Cupos arriba responde 404, que no se guarda en caché, así Redis no oculta la caída.

Fase	HTTP	Muestras	Latencia
0 a 20 s tras detener Cupos	504	30	mediana 2.010 ms (2.007–2.032)
Desde los 20 s	503	75	mediana 9,1 ms; p90 29 ms
Tras reiniciar Cupos	normal	5	recupera en 3,4–3,7 s (media 3,6 s)

Tabla 7: Respuesta de la API con Cupos detenido (5 repeticiones).

Resultados. El paso de 504 a 503 ocurrió a los **20,0 s en las 5 repeticiones**.

Interpretación. La hipótesis se cumple solo en parte. Al detener el contenedor, los paquetes hacia Cupos dejan de recibir respuesta y gRPC sigue intentando conectar. La documentación de gRPC fija en 20 s el tiempo mínimo que se le da a cada intento de conexión (`MIN_CONNECT_TIMEOUT`) [13]. Mientras el canal está en estado `CONNECTING`, las llamadas no fallan: esperan [14]. Por eso cada una agota el deadline y la API responde 504 en 2 s. Cuando el intento de conexión vence, el canal pasa a `TRANSIENT_FAILURE` y las llamadas fallan de inmediato con `UNAVAILABLE: 503` en milisegundos. La coincidencia exacta con los 20 s documentados respalda esta explicación.

Conclusiones y límites.

- **El deadline cumplió su función:** ninguna respuesta superó los 2,04 s. Sin deadline, las llamadas de los primeros 20 s esperarían el intento de conexión completo; esto es una inferencia de la documentación, no se midió.
- El costo: durante 20 s cada petición retiene un hilo de Matrículas por 2 s. Con carga alta, eso podría agotar el pool de hilos (no medido).
- **El ADR-004 original afirmaba “Cupos caído → 503”; la medición mostró que primero hay 20 s de 504.** Se corrigieron el ADR, el README y este informe. Ambos códigos están declarados en el contrato y ambos son reintentables.
- Límites: se probó solo la detención del contenedor. Si muriera el proceso de Cupos dentro de un contenedor vivo, la conexión se rechazaría de inmediato y probablemente se vería 503 desde el inicio (no medido). No se probó un Cupos *lento*, que produciría 504 sostenidos. Una sola máquina (Docker Desktop sobre Windows).

11. Instrucciones de ejecución

Requisito: Docker con Docker Compose. Desde la raíz del repositorio:

```
docker compose up --build
```

La API queda en `http://localhost:8000`, con documentación navegable en `/docs` y el contrato en `/openapi.yaml`. Tokens de desarrollo: `desarrollo-seguro` (lectura y escritura) y `solo-lectura` (solo GET). Ejemplo de uso:

```
curl -X POST http://localhost:8000/v1/estudiantes \
-H "Authorization: Bearer desarrollo-seguro" \
-H "Content-Type: application/json" \
-d '{"nombre": "Ana Perez", "email": "ana@example.com"}'
```

```
curl -X POST http://localhost:8000/v1/matriculas \  
-H "Authorization: Bearer desarrollo-seguro" \  
-H "Content-Type: application/json" \  
-H "Idempotency-Key: 7f1c-intento-1" \  
-d '{"estudiante_id": "<id devuelto>", "curso_id": "ARQ-101"}'
```

Otras operaciones:

```
docker compose stop cupos          # demostrar el modo de falla (T7)  
docker compose start cupos  
docker compose --profile demo run --rm cupos-node-client  # cliente Node (05)  
  
pip install -r requirements-dev.txt  
python -m unittest discover -s tests -v                      # pruebas (04)  
python docs/experimento_serializacion.py  
python docs/experimento_resiliencia.py --repeticiones 5 --duracion 35
```

Los cursos de ejemplo (ARQ-101, API-201, DAT-110) se cargan al iniciar Cupos. Para partir con las bases vacías: `docker compose down -v`.

12. Resumen de decisiones descartadas

Nivel de decisión	Se eligió	Se descartó
Organización interna	Microservicios (2 servicios, BD propia)	Monolito, monolito en capas, hexagonal como reemplazo de la separación, microservicios con BD compartida
Estilo de integración	Invocación remota	Transferencia de archivos, base de datos compartida, mensajería
Gobernanza de la integración	Descentralizada, contratos livianos	SOA con ESB, bus centralizado, WSDL/SOAP
Protocolo externo	REST/JSON versionado, contract-first	gRPC expuesto directamente, REST sin versión, contrato generado desde el código
Protocolo interno	gRPC/Protobuf, unario + server streaming	REST/JSON interno, client streaming, bidireccional
Caché	Cache-aside con invalidación, solo para lecturas	Sin caché, write-through desde Cupos, decidir matrículas con la caché
Reintentos duplicados	Idempotent Receiver (Idempotency-Key con SET NX)	Sin idempotencia, deduplicación por contenido
Colecciones	Paginación por desplazamiento	Colección completa, paginación por cursor
Ante falla de Cupos	Fail-fast con deadline (504, luego 503)	Esperar sin deadline, servir caché obsoleta como autorización
Autenticación	Bearer con tokens opacos por rol (401/403)	Basic Auth, API Key en header propio, JWT (por ahora)

Tabla 8: Matriz consolidada de alternativas y decisiones.

13. Reflexión

La separación protege la propiedad de los datos y permite escalar Cupos de forma independiente. El costo es que una operación de negocio atraviesa la red: por ello los contratos, deadlines, códigos de error y pruebas de caída son parte del diseño, no detalles posteriores. Lo que se buscó no fue la opción “mejor” en abstracto, sino la que mejor responde a las restricciones reales de AprendeMás: dos equipos con datos de naturaleza distinta, una consulta de disponibilidad de alto volumen, y una necesidad de respuesta inmediata que descarta tanto la mensajería como la base de datos compartida.

13.1. Qué haríamos distinto

- **Escribir el contrato antes del código y probarlo desde el inicio.** Durante un tiempo convivieron dos contratos OpenAPI: el escrito a mano y el que FastAPI generaba desde el código. Se desalinearon sin que nadie lo notara, hasta que las pruebas de contrato validaron las respuestas reales contra el YAML.
- **Medir el modo de falla antes de documentarlo.** El ADR-004 afirmaba “Cupos caído → 503”; al medirlo aparecieron 20 s de 504. El comportamiento depende de cómo falla la

dependencia y de la librería del cliente, no solo de nuestro código.

- **No versionar código generado.** Los stubs del `.proto` estaban copiados en dos servicios y el `.proto` en dos carpetas; cualquier cambio del contrato habría dejado a un consumidor desalineado.

13.2. Qué quedó pendiente

- **Identidad y cifrado:** JWT de vida corta con el rol como claim, TLS delante de la API y en el canal gRPC (hoy `insecure_channel`).
- **Consistencia:** conciliación periódica entre matrículas activas y cupos ocupados, para detectar cupos huérfanos (§3.3).
- **Resiliencia:** acortar la fase de 504 ajustando la reconexión del canal o con un circuit breaker, y medir con Cupos lento y bajo carga concurrente.
- **Concurrencia en Matrículas:** hoy usa una sola conexión SQLite compartida entre hilos, suficiente para la demo pero no para carga real.
- **Integración continua** que ejecute las pruebas de contrato en cada cambio.

Referencias

- [1] G. Hohpe y B. Woolf. *Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions*. Addison-Wesley, 2003.
- [2] S. Newman. *Building Microservices*, 2.^a ed. O'Reilly, 2021.
- [3] M. Fowler y J. Lewis. *Microservices*, 2014. <https://martinfowler.com/articles/microservices.html>
- [4] E. Evans. *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley, 2003.
- [5] M. Fowler. *Richardson Maturity Model*, 2010. <https://martinfowler.com/articles/richardsonMaturityModel.html>
- [6] R. Fielding, M. Nottingham y J. Reschke. *RFC 9110: HTTP Semantics*. IETF, 2022.
- [7] M. Nottingham, E. Wilde y S. Dalal. *RFC 9457: Problem Details for HTTP APIs*. IETF, 2023.
- [8] M. Jones y D. Hardt. *RFC 6750: The OAuth 2.0 Authorization Framework: Bearer Token Usage*. IETF, 2012.
- [9] E. Wilde. *RFC 8594: The Sunset HTTP Header Field*. IETF, 2019.
- [10] OpenAPI Initiative. *OpenAPI Specification 3.0.3*. <https://spec.openapis.org/oas/v3.0.3>
- [11] Google. *Protocol Buffers: Language Guide (proto3)*. <https://protobuf.dev/programming-guides/proto3/>
- [12] gRPC Authors. *Deadlines*. <https://grpc.io/docs/guides/deadlines/>

- [13] gRPC Authors. *gRPC Connection Backoff Protocol*. <https://github.com/grpc/grpc/blob/master/doc/connection-backoff.md>
- [14] gRPC Authors. *Wait-for-Ready Semantics*. <https://github.com/grpc/grpc/blob/master/doc/wait-for-ready.md>
- [15] Redis. *SET command*. <https://redis.io/docs/latest/commands/set/>
- [16] G. Pérez Correa. *Integración de Sistemas, semanas 2 a 4* [diapositivas de clase]. Universidad de Concepción, 2026.

A. Registros de decisión (ADR)

Versión condensada de los ADR de `docs/adr/`, con la plantilla del encargo.

ADR-001 · Frontera entre Cupos y Matrículas (D1) — aceptada

Contexto. Hay sobrecupos porque Matrículas inscribe sin consultar a Cupos. Los dos sistemas ya existen por separado, con dueños y ritmos de cambio distintos. Cupos es la fuente de verdad de los cupos; Matrículas guarda datos personales.

Alternativas. Monolito con base compartida (transacción local, pero fusiona dos sistemas con equipos distintos). Dos servicios con base compartida (monolito distribuido). Dos servicios con base propia y frontera por bounded context (autonomía, a cambio de una llamada remota y sin ACID entre ambos).

Decisión. Dos servicios con base propia: Cupos es dueño de `Curso` y sus contadores; Matrículas, de `Estudiante` y `Matrícula`.

Justificación. La frontera sigue la capacidad de negocio. La regla de cupo vive en un solo lugar: `OcuparCupo` verifica y descuenta de forma atómica. Se duplican a propósito solo el `curso_id` y una copia de lectura en caché que nunca decide matrículas.

Costo aceptado. Sin transacción entre ocupar el cupo y guardar la matrícula. Un fallo intermedio puede dejar un cupo huérfano; se mitigan la duplicación concurrente (`índice único + LiberarCupo`) y la reversión concurrente (`UPDATE` condicional).

Consecuencias. Matrículas depende de la disponibilidad de Cupos (ADR-004). En producción: identificador de operación y conciliación periódica.

ADR-002 · REST hacia afuera, gRPC hacia adentro (D2) — aceptada

Contexto. La API la consumen personas y un futuro portal web (incluidos navegadores); la llamada a Cupos es interna y ocurre en cada intento de matrícula.

Alternativas. REST en ambos lados (simple, sin contrato compilado). gRPC en ambos lados (el navegador necesitaría gRPC-Web y un proxy, y se expondría el puerto interno). REST afuera y gRPC adentro.

Decisión. REST/JSON bajo /v1 hacia afuera; gRPC/protobuf hacia Cupos, con `ListarCursos` como server streaming.

Justificación. Contrato compilado entre servicios; 39 frente a 94 bytes por `Curso` (0,40 estable; 0,73–0,85 con gzip, §10.1); interoperabilidad demostrada con el cliente Node.

Costo aceptado. Dos contratos, dos formatos y dos juegos de herramientas de depuración; gRPC no se inspecciona con curl.

Consecuencias. El puerto de Cupos no se publica; el cliente Node corre dentro de la red de Compose. Los stubs se generan en el build desde el único `.proto`.

ADR-003 · Contratos, versionado y evolución (D3) — aceptada

Contexto. Los contratos tienen consumidores que el equipo no controla del todo. Hay que saber qué cambios son compatibles y cómo se entera cada consumidor.

Alternativas. Sin versión y solo cambios aditivos; versión en cabecera; versión en la ruta y evolución aditiva en protobuf.

Decisión. /v1 en la ruta y `openapi.yaml` escrito a mano como contrato publicado; `package cupos.v1` con evolución aditiva de números de campo.

Justificación. Si mañana hay que agregar un campo (por ejemplo `cupos_totales` en la respuesta de curso): en REST es compatible, porque el cliente ignora lo que no conoce; en protobuf, se usa un número nuevo (= 5) y los clientes viejos lo ignoran. Exigen versión nueva: renombrar o eliminar campos en REST, cambiar tipos o significados, volver obligatorio un parámetro. En protobuf, eliminar un campo exige marcar su número como **reserved**. La paginación se incluyó desde /v1 porque agregarla después sería incompatible.

Costo aceptado. Mantener /v1 mientras exista /v2; más trabajo inicial al escribir el contrato a mano.

Consecuencias: cómo se enteran los consumidores. El contrato se revisa en cada pull request, se publica en `/openapi.yaml`, las pruebas de contrato fallan ante un campo no declarado, y al deprecarse /v1 sus respuestas llevarían la cabecera `Sunset` [9] con la fecha de retiro, registrando antes quién la sigue usando.

ADR-004 · Fallas de Cupos y de Redis (D4) — aceptada

Contexto. Si Cupos cae o se pone lento, la API no puede quedarse colgada ni matricular sin validar. Si Redis cae, la API no debería caerse por una optimización.

Alternativas. Esperar sin deadline (arrastra a Matrículas); aceptar con la caché (reabre el sobre-cupo); fail-fast con deadline.

Decisión. Deadline de 2 s por llamada. Traducción: UNAVAILABLE → 503, DEADLINE_EXCEEDED → 504, NOT_FOUND → 404, FAILED_PRECONDITION → 409 y cualquier otro código → 502, todos en `problem+json` sin detalles internos. Redis opcional para la caché; si falla con `Idempotency-Key`, 503. Tras una falla de Redis se deja de intentarlo durante 15 s.

Justificación. Medido (§10.2): con Cupos detenido, 504 en 2 s durante los primeros 20,0 s (5/5 repeticiones, igual al `MIN_CONNECT_TIMEOUT` de gRPC) y luego 503 en ~9 ms; recuperación en 3,4–3,7 s. Ninguna respuesta superó los 2,04 s. Con Redis detenido, resolver su nombre tarda 4–8 s y el timeout de conexión no lo cubre; sin la pausa de 15 s cada petición sumaba esa espera (hasta 11,7 s), con la pausa solo la primera.

Costo aceptado. No se matricula mientras Cupos no responde; durante 20 s cada petición retiene un hilo 2 s; sin Redis, las peticiones con clave reciben 503, cada 15 s una petición vuelve a probar Redis y paga la espera, y al volver Redis la caché puede tardar hasta 15 s en reactivarse.

Consecuencias. Los clientes reintentan 503 y 504 con la misma clave. Pendiente: acortar la fase de 504 (reconexión o circuit breaker) y medir con Cupos lento.

ADR-005 · Autenticación y autorización — aceptada

Contexto. La API expone datos personales y operaciones que ocupan cupos. Hay que saber quién llama y distinguir entre consultar y matricular.

Alternativas. Basic Auth; API Key en header propio; JWT; Bearer con tokens opacos por rol.

Decisión. `Authorization: Bearer` con un token de escritura y uno de solo lectura. 401 con `WWW-Authenticate` si falta el token o es desconocido; 403 si es válido pero no alcanza. Comparación en tiempo constante.

Justificación. Un token opaco fijo funciona, en rigor, como una API Key; se envía como Bearer porque es el esquema estándar (RFC 6750) que declaran OpenAPI, Swagger UI y curl, define cómo responder el 401, y permite migrar a JWT sin cambiar el contrato. Con dos roles, el 403 tiene un caso real.

Costo aceptado. Tokens compartidos, no personales: no permiten auditar a la persona; si se filtran hay que rotarlos; sin TLS viajan en claro.

Consecuencias. En producción: JWT de vida corta emitidos por un proveedor de identidad, con el rol como claim, y TLS obligatorio.